```
# Understanding the structure of git configration file

-> We have three levels of Configuration
   1) System Level Configuration
   2) Local Level Configuration
   3) Global Level Configuration

-> Every level configration has configration files.
-> Every configration files is devided into sections
   section is defined by square bracket as [ ]
-> Every section has key-value paire
```

-> Sample structure of git configration file

```
[core]
	repositoryformatversion = 0
	filemode = false
	bare = false
	logallrefupdates = true
	ignorecase = true
[remote "origin"]
	url = https://github.com/RRDTNP2024/HRM.git
	fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
	remote = origin
	merge = refs/heads/main
```

This is called section
section name is specified
in square bracket

These are key value paire
under each section.

If we want to change any key value by using "git config" command then sytntax is

section_name.subsection_name.key

Example: If we want to access filemode of core section then
core.filemode
Command will become git config --global core.filemode false

Some important sections

[core] section and its key are as below

```
	repositoryformatversion = 0
	filemode = false
	bare = false
	logallrefupdates = true
	ignorecase = true
```

1) repositoryformatversion: represent format version of git repository
Usually its value is 0

2) filemode : its value is either true or false
file mode indicate whether git should tract executable bit of file.
filemode=true(for Linux)
filemode=false (for Windows)

3) bare : Its value is either true or false
bare means no working directory.
bare=true(for bare repository)
base=false (for normal repository)

4) logallrefupdates : Its value is either true or false.
all reference updates should be logged or not.
if value is true means references get logged.
If value is false means references should not be logged.
logallrefupdates =true
logallrefupdates =false

[user] section
-> user section has name and email
-> Format

```
[user]
      name="user name"
      email="email address
```

-> example

```
[user]
     email = rrd-tnp@gmail.com
     name = rrd-tnp
```

[remote "origin"] section

-> this section defines remote repository and its urls.

-> under this section we have "url" and "fetch"

-> url: defines url of remote repository

fetch: defines which references are fetched from remote

Example:

```
[remote "origin"]
	url = https://github.com/RRDTNP2024/HRM.git
	fetch = +refs/heads/*:refs/remotes/origin/*
```

```
# Hierarchy of Configuration files
1) System Level Configuration: Applies to every user on system.
2) Local Level Configuration: Applies to specific repository
3) Global Level Configuration: Applies to specific user.
-------------------------
# Precedence Order of Configuration
1. Local Setting overrides global setting
2. Global settings overrides system setting
----------------------------------
```

Common Daily Operations for DevOps Engineer using GIT

1) Cloning and Managing Repository
2) Creating branches(release,bug-fix,development,feature)
3) Merging and rebasing branches
4) Review the Pull request
5) commit and push the code changes
6) Configure git hooks for Automation
7) Integrating GIT with CI/CD pipelines.
8) Troubleshoot git issue.
9) Tagging release.